

TheraPose

José de Jesús Fernández García, Noé Ortega Sánchez

CENTRO UNIVERSITARIO DE CIENCIAS

EXACTAS E INGENIERÍAS, (CUCEI, UDG)

dejesus.fernandez@alumnos.udg.mx

noe.ortega@academicos.udg.mx

Abstract— TheraPose es una aplicación Python de escritorio para sistemas operativos Linux fácilmente distribuible utilizando Docker, el cual tiene como objetivo servir de apoyo en el área de la terapia física para llevar un control y registro de los pacientes, así como para su rehabilitación mediante la implementación de juegos 2D interactivos haciendo uso de dos cámaras USB. TheraPose busca ser apoyo en el proceso de rehabilitación de los pacientes con la intención de mejorar los resultados.

Palabras claves – Proyecto modular, terapia física, rehabilitación, juegos 2D interactivos, modelo de cámara Pinhole, MediaPipe.

I. INTRODUCCIÓN

Hoy en día la terapia física no cuenta con sistemas informáticos de fácil acceso que ayuden en el proceso de rehabilitación de los pacientes, y muchos de los que ya existen suelen ser bastante grandes, costosos [1], y por lo general, son muy específicos para ciertas patologías, por lo que no todos los pacientes pueden darle uso.

En la terapia física, la rehabilitación de los pacientes es un proceso de mucha constancia y de llevar a cabo ejercicio tras ejercicio, lo cual para algunos pacientes es tedioso y en ciertos casos aburrida.

La implementación de juegos interactivos en la rehabilitación del paciente es muy poco usada, a pesar de que estos pueden activar los sistemas de recompensa del cerebro, lo cual puede ayudar en el proceso de rehabilitación del paciente.

Los beneficios de TheraPose están fuertemente ligados con mejorar el proceso de rehabilitación de los pacientes, con el fin de que sea más ameno y menos aburrido, siempre buscando los mejores resultados para los pacientes. No busca suplir a los fisioterapeutas, pero sí que sea un apoyo para ellos.

TheraPose tiene como objetivos principales ser un sistema informático económico, de uso sencillo e interactivo con el paciente. Además, procura no ser invasivo.

En resumen, TheraPose no solo plantea una solución interactiva para el proceso de rehabilitación del paciente, si no que abre las posibilidades de crear sistemas informáticos económicos y eficientes que pueden ser de apoyo en ciertas áreas de la salud, como lo puede ser la terapia física.

II. TRABAJOS RELACIONADOS

TheraPose está inspirado en muchos otros trabajos relacionados con la rehabilitación física mediante juegos interactivos. Algunos de estos trabajos hacen uso de la consola Nintendo Wii [2] como herramienta para mejorar el equilibrio y la propiocepción. Algunos hacen uso del sensor Kinect [3,4] de la consola de Xbox para obtener información como la posición de las articulaciones y ángulos articulares mediante la captura de movimiento. Otros más hacen uso de la realidad virtual [5] para tratar lesiones de hombro. Varios hacen uso de sensores inerciales [6,7,8] colocados en una parte específica del cuerpo para medir algún tipo de señal muscular, para medir la marcha y el equilibrio, o en ocasiones para capturar el movimiento y trasladarlo a un sistema robotizado para la rehabilitación. Otros hacen uso de trajes especiales de captura de movimiento [9] (los cuales suelen ser invasivos) para obtener datos del movimiento en casos como la hemiplejía (parálisis de la mitad del cuerpo). Otros utilizan sensores inerciales y realidad virtual [10] para capturar el movimiento de la actividad física del miembro inferior. Y otros trabajos más que son comerciales son [11] y [12], los cuales hacen uso del sensor Kinect, sin embargo, estos productos a pesar de que ofrecen grandes soluciones y son muy precisos, suelen ser de alto costo. Por último, un trabajo también comercial que sirvió de inspiración para crear TheraPose es OpenCap [13], el cual, aunque no precisamente implementa juegos interactivos para la rehabilitación, tiene un enfoque de análisis de movimiento mediante el procesamiento de datos generados por video de dos celulares para la simulación musculoesquelética, en donde se recaban datos como fuerzas, ángulos articulares, entre otros, teniendo como limitante que no es en tiempo real.

Sin duda los trabajos antes mencionados dan buenos resultados en sus respectivos problemas a resolver. El único problema es que todos requieren de aparatos que, por lo general, no se tienen a la mano y resultan costosos de adquirir.

La solución que se propone con TheraPose es servir de apoyo para los fisioterapeutas, el cual proporciona control y registro de pacientes, y además usa visión estéreo (Fig. 1) mediante el uso de dos cámaras calibradas de bajo costo, de modelos de deep learning creados en el framework MediaPipe [14] y del modelo de cámara Pinhole para proporcionar estimaciones 3D de puntos característicos del cuerpo humano, los cuales una vez procesados sirven para interactuar con juegos 2D implementados para la rehabilitación del paciente.

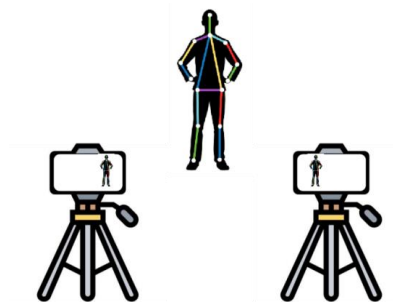


Fig. 1 La base de la visión estéreo se basa en la triangulación de rayos desde múltiples puntos de vista, la cual copia fielmente el funcionamiento de nuestros ojos para brindarnos una percepción de profundidad precisa. Elaboración propia

III. DESCRIPCIÓN DEL DESARROLLO DEL PROYECTO MODULAR

A. Hipótesis

En la terapia física, los resultados de los pacientes se pueden maximizar si se implementan juegos interactivos para su rehabilitación mediante el uso de un sistema informático económico que hace uso de dos cámaras USB para la interactividad.

B. Metodología

El desarrollo del proyecto se llevó a cabo siguiendo la metodología de desarrollo en espiral y consiste en una aplicación Python de escritorio desarrollada con el framework CustomTkinter, la cual hace uso de la base de datos no relacional MongoDB para el control y registro de pacientes.

El proyecto está pensado para no depender de la conexión a internet, por lo que se puede trabajar tanto con una base de datos local (imagen Docker: *mongo:5*) como con una base de datos en la nube (MongoDB Atlas).

La aplicación de escritorio y la base de datos local están encapsuladas de forma independiente por medio de Docker para su fácil distribución.

Su uso es específico para sistemas operativos Linux que cuenten con Docker y Docker-Compose.

La interacción con los juegos 2D para la rehabilitación se lleva a cabo por medio de dos cámaras de bajo costo, de modelos de deep learning para la estimación 2D de puntos característicos del cuerpo y del modelo de cámara Pinhole.

C. Materiales

Para el proyecto fueron necesarios los siguientes materiales:

- PC de escritorio o laptop
- 2 cámaras USB con resolución de video 480p
- 2 trípodes (opcional, la intención es que las cámaras estén fijas y puedan manipularse fácilmente)
- 2 patrones conocidos
 - Tablero de ajedrez
 - Marcador ArUco

Es importante que los patrones conocidos sean impresos y pegados en alguna tabla rígida para su posterior uso. Pueden

estar pegados en la misma tabla por lados diferentes. Los tipos y las medidas de dichos patrones serán muy importantes más adelante, por lo que se debe tomar en cuenta.

D. Patrones Conocidos

Los patrones son de suma importancia para varias tareas y pueden existir de muchos tipos y dimensiones. Los patrones que se usaron se muestran en la Fig. 2 y son usados en los siguientes casos:

- Tablero de ajedrez: Es usado en el proceso de calibración
- Marcador ArUco: Es usado en el proceso previo a la interacción con los juegos 2D

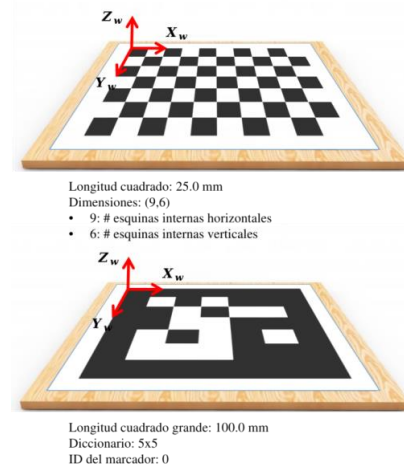


Fig. 2 Tablero de ajedrez y marcador ArUco, respectivamente. El sistema de coordenadas del mundo es el que está definido en cada uno de ellos (las unidades que se utilizan son los milímetros). Los datos en milímetros que se muestran no están a escala. Elaboración propia

E. Encapsulación del Proyecto con Docker

El proyecto está dividido en dos partes independientes, cada una de las cuales se puede distribuir fácilmente con Docker:

- Servicio aplicación Python
 - Apartado de rehabilitación: En este apartado se llevan a cabo tareas como la calibración de las cámaras y la interacción con los juegos 2D
 - Apartado de base de datos: En este apartado se lleva a cabo el control y registro de los pacientes
- Servicio base de datos local MongoDB
 - Configurada como un conjunto de replicas (clúster) utilizando 3 instancias de MongoDB
 - Imagen Docker para cada instancia: *mongo:5*

Cada uno de los servicios es lanzado por medio de Docker-Compose mediante un script bash interactivo que automatiza todo este proceso, así como también ayuda a detener dichos servicios. El script bash también realiza otras tareas que son necesarias cada vez que se inician los servicios. La interfaz de línea de comandos del script se muestra en la Fig. 3.

La estructura de carpetas y archivos para el proyecto en general es la siguiente, la cual es muy importante y se debe seguir para que el script de bash pueda iniciar los servicios:

- app (carpeta)
 - app (carpeta): Contiene todo el código de Python del proyecto
 - docker-compose.yaml (archivo): Archivo para lanzar el servicio de la aplicación Python
 - Dockerfile (archivo): Archivo para construir una imagen de Docker que contenga toda la aplicación de Python
 - requirements.txt (archivo): Archivo que especifica todas las dependencias necesarias para la aplicación de Python
- db (carpeta)
 - .env (archivo): Archivo para establecer variables de entorno para MongoDB
 - docker-compose.yaml (archivo): Archivo para lanzar el servicio de la base de datos local MongoDB
 - init_database.js (archivo): Archivo para crear un usuario diferente del root y para construir e inicializar la base de datos local con algunos registros
 - mongo_replication.key (archivo): Archivo necesario para configurar un conjunto de replicas en MongoDB
 - mongo_setup.sh (archivo): Script bash para configurar un grupo de instancias MongoDB en un conjunto de replicas
- therapose.sh (archivo): Script bash para iniciar y detener los servicios del proyecto



Fig. 3 Interfaz del script bash therapose.sh. Elaboración propia

F. Control y Registro de Pacientes

Anteriormente se mencionó que se puede trabajar tanto en la base de datos local como en la base de datos en la nube. Las ventajas de hacerlo de esta manera son las siguientes:

- El proyecto no depende de la conexión a internet
- Se tiene respaldo de la información tanto en local como en la nube

Las colecciones que se utilizan en la base de datos MongoDB son las siguientes:

- DatabaseUpdates: Para registrar la fecha en la que se ha guardado en la base de datos
- Session: Para registrar los posibles tipos de sesión

- CurrentSituation: Para registrar las posibles situaciones actuales
- Pathology: Para registrar patologías
- Disease: Para registrar enfermedades
- Patient: Para guardar todo lo relacionado a la información del paciente, tal como:
 - Nombre
 - Tipo de sesión
 - Situación actual
 - Patologías
 - Enfermedades
 - Agenda

Con respecto a las colecciones, Session y CurrentSituation son fijas; DatabaseUpdates, Pathology y Disease se actualizan por si solas cuando es necesario; y, por último, Patient es donde se realizan todo tipo de operaciones.

Todo el proceso que conlleva el control y registro de pacientes se muestra en la Fig. 4.

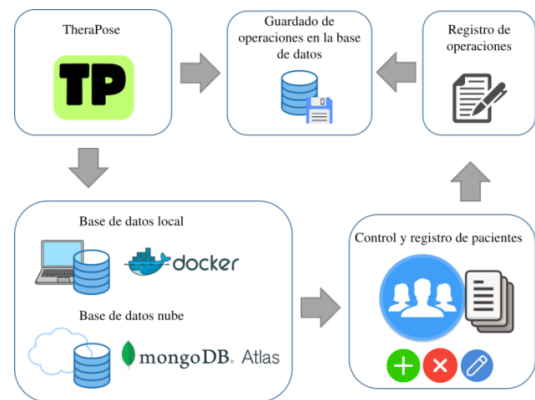


Fig. 4 Flujo de trabajo para el apartado de base de datos. Elaboración propia

Otros aspectos que se deben destacar son los siguientes:

- Es posible cargar la base de datos local/nube en la base de datos nube/local, con el objetivo de recuperar la información en caso de haberla perdido, ya sea en local o nube
- Tanto la base de datos local como la base de datos en la nube cuentan con un registro de operaciones, el cual se actualiza cada vez que se hacen cambios en los datos, esto independientemente de que base de datos se esté usando
- Se cuenta con guardado en automático del registro de operaciones en caso de que se cierre inesperadamente la aplicación Python (se guarda en un archivo en la propia computadora)
- Siempre se tienen los datos más actualizados independientemente de que base de datos se esté usando

G. Modelo de Cámara Pinhole

El modelo de cámara Pinhole (o estenopeica) es un modelo matemático que describe la relación matemática de un punto 3D y su proyección en el plano de imagen 2D de la cámara.

A menudo este modelo se puede utilizar como una descripción razonable de como una cámara representa una escena 3D, siempre y cuando no se usen cámaras muy complejas o que produzcan mucha distorsión.

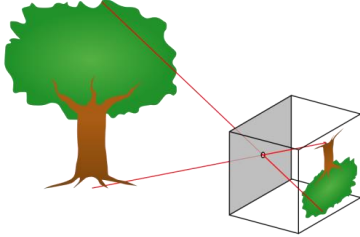


Fig. 5 Funcionamiento de una cámara Pinhole. El punto donde convergen todos los rayos de luz se le conoce como centro óptico, y es ahí donde se define el sistema de coordenadas de la cámara (sistema tridimensional). El eje óptico es normal al plano de imagen de la cámara y pasa por el centro óptico. [15]

El modelo de cámara Pinhole se puede dividir en dos partes:

- Modelo ideal: No toma en cuenta las distorsiones que produce la lente de la cámara
- Modelo no ideal: Toma en cuenta las distorsiones que produce la lente de la cámara

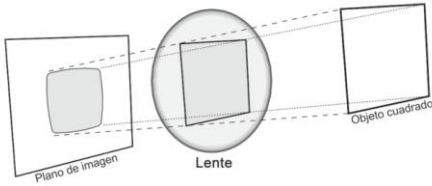


Fig. 6 Distorsión producida por la lente de una cámara. [16]

El modelo como tal está basado en los resultados que produce la Fig. 5. Estos resultados corresponden al modelo de cámara Pinhole ideal. Para el modelo de cámara Pinhole no ideal los resultados son algo diferentes, pero tienen un análisis similar, la única diferencia es que toma en cuenta la refracción que tienen los rayos de luz al pasar por la lente de la cámara, tal y como se muestra en la Fig. 6.

En la práctica realmente se utilizan los dos modelos, el ideal y el no ideal. Mas adelante se explicará en detalle cuando se usa uno y cuando se usa otro.

Los modelos matemáticos para cada uno de los modelos son los siguientes:

- Modelo de cámara Pinhole ideal

$$z_c m = K Q M \quad (1)$$

Donde:

- z_c : Es un escalar

$$m = \begin{bmatrix} v_{3p} \\ 1 \end{bmatrix}$$

$$K = \begin{bmatrix} f_x & -f_x \tan \theta & \alpha_x(t_{3y} \tan \theta - t_{3x}) \\ 0 & f_y \sec \theta & -\alpha_y t_{3y} \sec \theta \\ 0 & 0 & 1 \end{bmatrix}$$

Matriz intrínseca (K)

Relaciona ciertos parámetros de la cámara, como lo son:

- ◆ Distancia focal (f)
- ◆ Posición del sistema de coordenadas del plano de imagen de la cámara (es el sistema de coordenadas que al final se termina usando) visto desde la intersección del eje óptico con el plano de imagen (t_3)
- ◆ Angulo en radianes que mide el grado de no ortogonalidad entre los vectores base del sistema de coordenadas del plano de imagen de la cámara (θ)

$$Q = [T_{wc} \quad t_w]$$

Matriz extrínseca (Q)

Relaciona ciertos parámetros que no pertenecen a la cámara, como lo son:

- ◆ Matriz de rotación del sistema de coordenadas del mundo con respecto al sistema de coordenadas de la cámara (T_{wc})
- ◆ Vector de traslación del sistema de coordenadas del mundo con respecto al sistema de coordenadas de la cámara (t_w)

$$M = \begin{bmatrix} v_w \\ 1 \end{bmatrix}$$

- Modelo de cámara Pinhole no ideal

$$m = K^* v^* \quad (2)$$

Donde:

$$m = \begin{bmatrix} v_{3p} \\ 1 \end{bmatrix}$$

$$K^* = \begin{bmatrix} -f_x & f_x \tan \theta & \alpha_x(t_{3y} \tan \theta - t_{3x}) \\ 0 & -f_y \sec \theta & -\alpha_y t_{3y} \sec \theta \\ 0 & 0 & 1 \end{bmatrix}$$

Es posible construir K^* a partir de K

$$v^* = \begin{bmatrix} v_1^{**} \\ 1 \end{bmatrix}$$

A continuación se muestra más detalle de las variables presentes en los diferentes modelos:

$$v_{3p} = \begin{bmatrix} x_{3p} \\ y_{3p} \end{bmatrix} \text{ (píxeles)}$$

Vector con respecto al sistema de coordenadas del plano de imagen de la cámara

$$f_x = f \alpha_x \text{ (píxeles)}$$

$$f_y = f \alpha_y \text{ (píxeles)}$$

$$\text{Distancia focal: } f \text{ (mm)}$$

$$\text{Factor de conversión (mm a píxeles) en dirección del eje X: } \alpha_x \left(\frac{\text{píxeles}}{\text{mm}} \right)$$

- Factor de conversión (mm a pixeles) en dirección del eje Y: $\alpha_y (\frac{\text{pixeles}}{\text{mm}})$

- $t_3 = \begin{bmatrix} t_{3x} \\ t_{3y} \end{bmatrix} (mm)$

- $\theta (rad)$

Por lo general es muy pequeño, y en ciertos casos despreciable

- $T_{wc} = [i_{wc} \ j_{wc} \ k_{wc}] = \begin{bmatrix} i_{wcx} & j_{wcx} & k_{wcx} \\ i_{wcy} & j_{wcy} & k_{wcy} \\ i_{wcz} & j_{wcz} & k_{wcz} \end{bmatrix}$

- $t_w = \begin{bmatrix} t_{wx} \\ t_{wy} \\ t_{wz} \end{bmatrix} (mm)$

- $v_w = \begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix} (mm)$

Vector con respecto al sistema de coordenadas del mundo

- $v_1^* = v_1^* + d(v_1^*)$

- $v_1^* = \begin{bmatrix} x_1^* \\ y_1^* \end{bmatrix} = \begin{bmatrix} -\frac{x_c}{z_c} \\ -\frac{y_c}{z_c} \end{bmatrix}$

- $v_c = \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix} = [T_{wc} \ t_w] \begin{bmatrix} v_w \\ 1 \end{bmatrix} (mm)$

Vector con respecto al sistema de coordenadas de la cámara

- $d(v_1^*) = \begin{bmatrix} d_x(v_1^*) \\ d_y(v_1^*) \end{bmatrix}$

Vector de distorsión

- $\begin{bmatrix} d_x(v_1^*) \\ d_y(v_1^*) \end{bmatrix} = \begin{bmatrix} x_1^*(k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 x_1^* y_1^* + p_2(r^2 + 2(x_1^*)^2) \\ y_1^*(k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2(y_1^*)^2) + 2p_2 x_1^* y_1^* \end{bmatrix}$

- $r = \|v_1^*\| = \sqrt{(x_1^*)^2 + (y_1^*)^2}$

■ Parámetros de distorsión:

◆ Radial: k_1, k_2, k_3

◆ Tangencial: p_1, p_2

◆ Como vector: $q = \begin{bmatrix} k_1 \\ k_2 \\ k_3 \\ p_1 \\ p_2 \end{bmatrix}$

En algunas ecuaciones se hace uso de notación por bloques.

H. Calibración de Cámara

La calibración de la cámara es un paso muy importante para poder interactuar con los juegos implementados para la rehabilitación, el cual consiste en calcular los parámetros intrínsecos y de distorsión de la cámara, y tan solo requiere que se haga una sola vez por cámara.

De las técnicas clásicas de calibración se tienen las siguientes:

- Direct Linear Transformation (DLT): Requiere de un equipo de calibración 3D (se requieren coordenadas 3D extremadamente precisas)

- Multiplane Calibration (método de Zhang): Requiere capturar múltiples vistas de una única superficie plana. El método que se utilizó está basado en el método de Zhang [17].

Los parámetros de la cámara que se desean estimar son los siguientes:

- Parámetros intrínsecos (K)
- Parámetros de distorsión (q)

Los pasos que se siguen en el proceso de calibración son los siguientes:

- 1) Haciendo uso del modelo de cámara Pinhole ideal (ecu. 1), se manipula hasta obtener la siguiente ecuación:

- a) Con $z_w = 0$ se tiene lo siguiente:

$$\begin{aligned} QM &= [T_{wc} \ t_w] \begin{bmatrix} v_w \\ 1 \end{bmatrix} \\ &= [i_{wc} \ j_{wc} \ k_{wc} \ t_w] \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix} \\ &= [i_{wc} \ j_{wc} \ k_{wc} \ t_w] \begin{bmatrix} x_w \\ y_w \\ 0 \\ 1 \end{bmatrix} \\ &= [i_{wc} \ j_{wc} \ t_w] \begin{bmatrix} x_w \\ y_w \\ 1 \end{bmatrix} \\ &= Q^* M^* \end{aligned}$$

Con esto indicamos que los puntos 3D únicamente viven en el plano $X_w Y_w$, tal y como se muestra en la Fig. 7 para los puntos definidos en el sistema de coordenadas del mundo.

- b) Por tanto:

$$z_c m = K Q^* M^* \quad (3)$$

- c) Se define lo siguiente:

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} = K Q^* \quad (4)$$

- d) Por tanto:

$$z_c m = H M^* \quad (5)$$

- e) Se multiplica a ambos lados de la ecuación por $\frac{1}{h_{33}}$ y se definen algunas nuevas variables:

$$\frac{1}{h_{33}} z_c m = \frac{1}{h_{33}} H M^* \quad (6)$$

Se establece lo siguiente:

$$\alpha = \frac{1}{h_{33}} z_c \quad (7)$$

$$\begin{aligned} G &= \frac{1}{h_{33}} H \\ &= \begin{bmatrix} g_{11} & g_{12} & g_{13} \\ g_{21} & g_{22} & g_{23} \\ g_{31} & g_{32} & 1 \end{bmatrix} \end{aligned} \quad (8)$$

f) Ecuación final resultante:

$$am = GM^* \quad (9)$$

- 2) Ahora, partiendo de la ecuación anterior, es posible calcular la matriz G mediante pares de puntos correspondientes 2D (en pixeles) y 3D (en mm), así como se muestra en la Fig. 7. Por cada par de puntos correspondientes se obtienen 2 ecuaciones, por tanto, para calcular a la matriz G (la cual tiene 8 incognitas) se necesitan por lo menos 4 pares de puntos correspondientes. El problema se puede plantear y resolver de la siguiente manera:

Problema: $Ax = b$

Solución: $x = (A^T A)^{-1} A^T b$

Donde:

- m : Numero de pares de puntos correspondientes ($m \geq 4$)
- $A \in R^{2m \times 8}$
- $b \in R^{2m \times 1}$
- $x = \begin{bmatrix} g_{11} \\ g_{12} \\ g_{13} \\ g_{21} \\ g_{22} \\ g_{23} \\ g_{31} \\ g_{32} \end{bmatrix} \in R^{8 \times 1}$

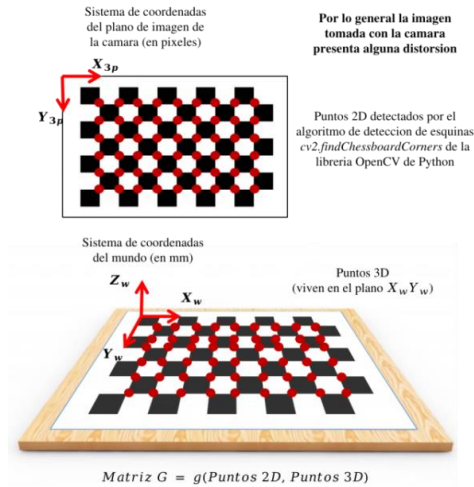


Fig. 7 Obtención de pares de puntos correspondientes 2D y 3D a partir de un tablero de ajedrez. Elaboración propia

- 3) Ya obtenida la matriz G se procede a construir las siguientes ecuaciones:

NOTA: Es importante mencionar que la matriz T_{wc} esta formado por un conjunto de vectores ortonormales, es decir, los vectores base del sistema de coordenadas del mundo con respecto al sistema de coordenadas de la cámara son ortonormales entre sí. Esto es:

$$\|i_{wc}\|^2 = \|j_{wc}\|^2 = \|k_{wc}\|^2 = 1 \quad (10)$$

$$i_{wc}^T j_{wc} = 0 \quad (11)$$

$$i_{wc}^T k_{wc} = 0 \quad (12)$$

$$j_{wc}^T k_{wc} = 0 \quad (13)$$

- a) Extendiendo la equ. 8 y utilizando la equ. 4 se tiene lo siguiente:

$$\begin{aligned} [g_1 \ g_2 \ g_3] &= \frac{1}{h_{33}} K [i_{wc} \ j_{wc} \ t_w] \\ &= \left[\frac{1}{h_{33}} K i_{wc} \quad \frac{1}{h_{33}} K j_{wc} \quad \frac{1}{h_{33}} K t_w \right] \end{aligned}$$

- b) Las ecuaciones resultantes son las siguientes:

$$\begin{aligned} g_1 &= \frac{1}{h_{33}} K i_{wc} \\ i_{wc} &= h_{33} K^{-1} g_1 \end{aligned} \quad (14)$$

$$\begin{aligned} g_2 &= \frac{1}{h_{33}} K j_{wc} \\ j_{wc} &= h_{33} K^{-1} g_2 \end{aligned} \quad (15)$$

$$\begin{aligned} g_3 &= \frac{1}{h_{33}} K t_w \\ t_w &= h_{33} K^{-1} g_3 \end{aligned} \quad (16)$$

- c) Utilizando las ecuaciones anteriores y sabiendo que T_{wc} esta formado por un conjunto de vectores ortonormales, se llega a las siguientes ecuaciones:

$$\begin{aligned} g_1^T (K^{-1})^T K^{-1} g_1 \\ - g_2^T (K^{-1})^T K^{-1} g_2 &= 0 \end{aligned} \quad (17)$$

$$g_1^T (K^{-1})^T K^{-1} g_2 = 0 \quad (18)$$

- d) De las ecuaciones anteriores, se define lo siguiente:

$$\begin{aligned} B &= (K^{-1})^T K^{-1} \\ &= \begin{bmatrix} a & b & c \\ b & d & e \\ c & e & f \end{bmatrix} \end{aligned} \quad (19)$$

Como propiedad importante a destacar de la matriz B es que es una matriz simétrica, es decir, $B = B^T$.

Notar que si se encuentra la matriz B entonces es posible encontrar la matriz intrínseca K de la cámara con algunos pasos adicionales. Esto se verá a continuación.

- e) Por tanto, las ecuaciones quedan de la siguiente manera:

$$\begin{aligned} g_1^T B g_1 - g_2^T B g_2 &= 0 \\ g_1^T B g_2 &= 0 \end{aligned}$$

- f) Las ecuaciones anteriores generan 2 ecuaciones para 6 incógnitas de la matriz B , por lo que para encontrar a la matriz B se necesitan por lo menos 3 matrices G . Por tanto, se deben tomar varias imágenes del tablero de ajedrez en diferentes orientaciones para obtener diferentes matrices G , tal cual como se explico anteriormente y como se muestra en la Fig. 7. Toda la información que se obtiene para calcular las matrices G (pares de puntos correspondientes para cada imagen) se debe guardar porque más adelante se va a necesitar para el proceso de refinamiento de los parámetros de la cámara.
- g) Las ecuaciones resultantes para cada matriz G se agregan para formar un sistema de ecuaciones más grande para poder determinar a la matriz B . El problema se puede plantear y resolver de la siguiente manera:

Problema: $Ax = 0$

Solución: Es el vector propio con el valor propio asociado más pequeño del siguiente problema:

$$A^T A x = \lambda x \rightarrow C x = \lambda x.$$

Donde:

- p : Numero de matrices G ($p \geq 3$)
- $A \in R^{2p \times 6}$

$$\bullet \quad x = \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \end{bmatrix} \in R^{6 \times 1}$$

- h) Ya obtenida la matriz B se procede a calcular la matriz intrínseca K de la cámara haciendo uso de la descomposición de Cholesky:

La descomposición de Cholesky permite descomponer una matriz simétrica definida positiva en la multiplicación de una matriz triangular inferior y la transpuesta de la matriz triangular inferior. No se asegura que la matriz B sea siempre una matriz definida positiva, por lo que si falla la descomposición de Cholesky

sobre B , entonces se deberá volver a repetir el proceso de captura de imágenes del tablero de ajedrez y todos los pasos mencionados anteriormente hasta llegar a este punto de nuevo. Tal descomposición queda de la siguiente manera:

$$\text{Cholesky}(B) = LL^T \quad (20)$$

Si $L = (K^{-1})^T$ entonces:

$$LL^T = (K^{-1})^T K^{-1} = B \quad (21)$$

Por tanto:

$$\begin{aligned} K &= (L^T)^{-1} \\ &= \begin{bmatrix} k_{11} & k_{12} & k_{13} \\ 0 & k_{22} & k_{23} \\ 0 & 0 & k_{33} \end{bmatrix} \end{aligned} \quad (22)$$

Es importante mencionar que la matriz K calculada no es exactamente la verdadera, esto es debido a que se utilizó el modelo de cámara Pinhole ideal en una cámara que, por lo general, no es ideal. Pero no es de alarmar, ya que más adelante se encontrarán los parámetros que realmente corresponden a los de la matriz K verdadera (se puede decir que la matriz K calculada es una aproximación a la matriz K verdadera).

- 4) Ya finalizado el proceso de la obtención de la matriz intrínseca K de la cámara, lo que sigue es calcular las matrices extrínsecas Q 's para cada matriz G , esto es, la matriz extrínseca Q para cada imagen tomada del tablero de ajedrez en diferentes orientaciones, lo cual es posible debido a la equ. 14, 15 y 16.

Se sabe que:

$$Q = [T_{wc} \quad t_w] = [i_{wc} \quad j_{wc} \quad k_{wc} \quad t_w]$$

Como T_{wc} tiene un conjunto de vectores ortonormales, entonces:

$$\|i_{wc}\| = \|j_{wc}\| = \|k_{wc}\| = 1 \quad (23)$$

Operando la ecuación anterior se llega a lo siguiente:

$$\begin{aligned} \|i_{wc}\| &= \|h_{33}K^{-1}g_1\| = h_{33}\|K^{-1}g_1\| = 1 \\ \|j_{wc}\| &= \|h_{33}K^{-1}g_2\| = h_{33}\|K^{-1}g_2\| = 1 \end{aligned}$$

$$h_{33} = \frac{1}{\|K^{-1}g_1\|} = \frac{1}{\|K^{-1}g_2\|} \quad (24)$$

Por tanto, utilizando la ecuación anterior en la equ. 14, 15 y 16, se tiene lo siguiente:

$$\begin{aligned} i_{wc} &= h_{33}K^{-1}g_1 = \frac{1}{\|K^{-1}g_1\|} K^{-1}g_1 \\ j_{wc} &= h_{33}K^{-1}g_2 = \frac{1}{\|K^{-1}g_1\|} K^{-1}g_2 \\ k_{wc} &= -(i_{wc} \times j_{wc}) \end{aligned}$$

$$t_w = h_{33}K^{-1}g_3 = \frac{1}{\|K^{-1}g_1\|}K^{-1}g_3$$

La operación que se realiza para calcular k_{wc} es un producto vectorial siguiendo la regla de la mano derecha. Todo esto está pensado para que $T_{wc} = [i_{wc} \ j_{wc} \ k_{wc}]$ coincida con el sistema de coordenadas del mundo.

Todo lo anterior es para calcular una matriz Q , por lo que se debe hacer el mismo procedimiento para cada matriz G que se haya calculado. Como resultado final se deben tener todas las matrices Q calculadas.

- 5) En este punto ya se tiene calculada la matriz intrínseca K de la cámara y las matrices extrínsecas Q 's asociadas a cada imagen tomada del tablero de ajedrez. Ahora lo que sigue es el proceso de refinamiento de los parámetros de la cámara, el cual consiste en obtener los parámetros reales resolviendo un problema de optimización con restricciones en donde se requiere minimizar una función objetivo, la cual mide el error entre un punto 2D predicho por el modelo de cámara Pinhole no ideal y un punto 2D real dado por el algoritmo de detección de esquinas del tablero de ajedrez, el cual se menciona en la Fig. 7. La información de los puntos 2D reales que se usan como datos de referencia son los datos que se obtuvieron en la obtención de las matrices G . La función a minimizar es la siguiente:

$$f(K^{**}, q, Q_1^*, \dots, Q_p^*) = \sum_{i=1}^p \sum_{j=1}^m \|v_{3p(real)}^{(i,j)} - v_{3p(predicted)}^{(i,j)}\|^2 \quad (25)$$

Sujeta a (para cada matriz Q):

$$\|i_{wc}\|^2 = i_{wc}^T i_{wc} = 1 \quad (26)$$

$$\|j_{wc}\|^2 = j_{wc}^T j_{wc} = 1 \quad (27)$$

$$i_{wc}^T j_{wc} = 0 \quad (28)$$

Las restricciones son para asegurar que los vectores base de las matrices de rotación T_{wc} asociadas a cada matriz Q formen una base ortonormal de vectores.

La función lagrangiana queda de la siguiente manera:

$$L(K^{**}, q, Q_1^*, \dots, Q_p^*, \lambda_1^1, \lambda_1^2, \lambda_1^3, \dots, \lambda_1^p, \lambda_2^p, \lambda_3^p) = f(K^{**}, q, Q_1^*, \dots, Q_p^*) - \sum_{i=1}^p [\lambda_1^i (i_{wc}^{(i)T} i_{wc}^{(i)} - 1) + \lambda_2^i (j_{wc}^{(i)T} j_{wc}^{(i)} - 1) + \lambda_3^i (i_{wc}^{(i)T} j_{wc}^{(i)})] \quad (29)$$

Donde:

- p : Numero de imágenes tomadas del patrón de calibración en diferentes orientaciones
- m : Numero total de puntos 2D capturados del tablero de ajedrez (este depende de las dimensiones del tablero de ajedrez)

$$K^{**} = \frac{1}{k_{33}} K = \begin{bmatrix} k_{11}^{**} & k_{12}^{**} & k_{13}^{**} \\ 0 & k_{22}^{**} & k_{23}^{**} \\ 0 & 0 & 1 \end{bmatrix} \quad (30)$$

Esto ayuda al proceso de convergencia, ya que la matriz K que se calculó anteriormente no asegura que $k_{33} = 1$, y por la naturaleza de K se sabe que $k_{33} = 1$. Por tanto, K^{**} viene a suplir a la matriz K calculada anteriormente.

$$Q^* = [i_{wc} \ j_{wc} \ t_w] \quad (31)$$

$$q = \begin{bmatrix} k_1 \\ k_2 \\ k_3 \\ p_1 \\ p_2 \end{bmatrix} \quad (32)$$

Como valores iniciales:

- La matriz intrínseca (K) calculada anteriormente
 - Las matrices extrínsecas (Q 's) calculadas anteriormente
 - Los parámetros de distorsión (q) se inicializan en cero
 - Los multiplicadores de Lagrange (λ 's) se inicializan en cero
- 6) Para minimizar la función lagrangiana (función objetivo) se utiliza su gradiente y como optimizador se usa *Adam*, el cual es muy usado en redes neuronales como optimizador.
- 7) Al final del proceso de refinamiento se tendrán los siguientes parámetros refinados (optimizados):
- Matriz intrínseca K de la cámara (la real)
 - Parámetros de distorsión q
 - Matrices extrínsecas Q 's

Todos los parámetros obtenidos se deben guardar para su posterior uso. Los más importantes son la matriz K y los parámetros de distorsión q . Las matrices Q 's nos pueden ayudar a visualizar las orientaciones del tablero de ajedrez (del proceso de captura de imágenes) con respecto al sistema de coordenadas de la cámara, así como se muestra en la Fig. 8.

Planos del tablero de ajedrez (unidades: cm)
Se utilizaron 11 planos

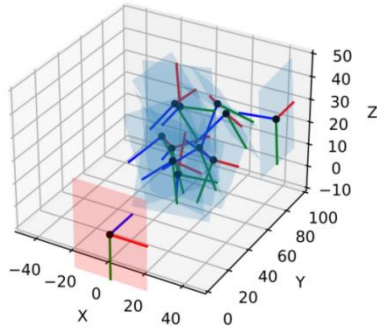


Fig. 8 Visualización de matrices extrínsecas calculadas en el proceso de calibración, en donde se tomaron 11 fotos del tablero de ajedrez en diferentes orientaciones. Elaboración propia

- 8) Como paso final, queda probar los resultados mediante realidad aumentada, en donde se dibuja un cubo sobre el tablero de ajedrez, así como se muestra en la Fig. 9. El uso de realidad aumentada pone a prueba la calidad de los parámetros de la cámara antes calculados.

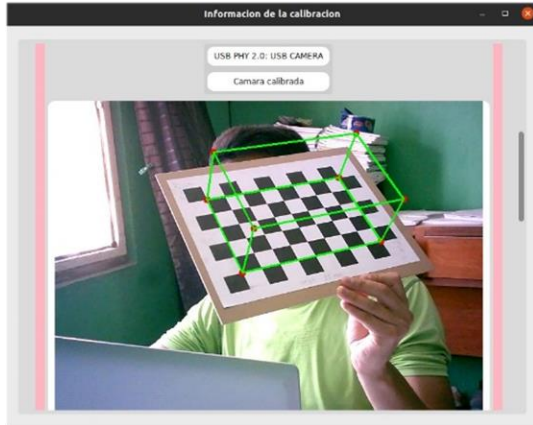


Fig. 9 Realidad aumentada utilizando los parámetros de la cámara calculados en el proceso de calibración. Elaboración propia

I. Estimaciones 3D

Es importante tener claro lo siguiente:

- Puntos 2D sin distorsión
 - Se usan en el modelo de cámara Pinhole ideal
- Puntos 2D con distorsión
 - Se usan en el modelo de cámara Pinhole no ideal
 - Los puntos 2D con distorsión pueden pasar a ser puntos 2D sin distorsión mediante un proceso que elimina la distorsión, por lo que dichos puntos 2D sin distorsión pueden pasar a ser usados en el modelo de cámara Pinhole ideal

Para realizar las estimaciones 3D se utilizan los modelos de deep learning MediaPipe Pose [18] y MediaPipe Hands [19], y los modelos ideal y no ideal del modelo de cámara Pinhole. Los modelos de deep learning son los encargados de detectar puntos 2D característicos del cuerpo humano en una imagen, tal y como se muestra en la Fig. 10.

Consideraciones que se deben tomar en cuenta:

- Las imágenes capturadas por la cámara tienen distorsión
- Para los algoritmos de detección de puntos 2D en una imagen, los puntos están distorsionados, por lo que corresponden al modelo no ideal
- Todos los puntos 2D distorsionados deberán ser procesados para obtener puntos 2D no distorsionados

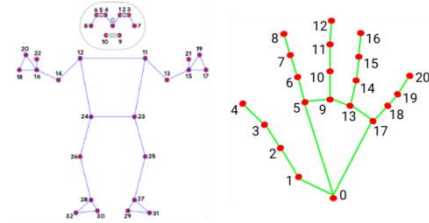


Fig. 10 Puntos característicos detectados por los modelos MediaPipe Pose y MediaPipe Hands, respectivamente. [18,19]

Para el proceso de eliminar la distorsión de puntos 2D distorsionados se realiza lo siguiente:

- a) Se utiliza el modelo de cámara Pinhole no ideal, ya que un punto 2D en la imagen tomada por la cámara es un punto 2D en dicho modelo (se corresponden).
- b) En base a dicho modelo y a su análisis, se define un problema de búsqueda de raíces de la siguiente función objetivo:

$$f(v_1^*) = \begin{bmatrix} v_{3p} \\ 1 \end{bmatrix} - K^* \begin{bmatrix} v_1^* + d(v_1^*) \\ 1 \end{bmatrix} \quad (33)$$

Se está interesado en encontrar v_1^* , ya que es el punto no distorsionado en el plano de imagen normalizado (dicho plano es paralelo al plano de imagen de la cámara y está a una distancia focal de 1 del centro optico). Por tanto, este punto sirve para calcular el punto 2D no distorsionado en el plano de imagen de la cámara.

Como valor inicial de v_1^* se puede usar v_1^{**} , ya que:

$$v_1^{**} = v_1^* + d(v_1^*) \quad (34)$$

Por lo que se puede asumir lo siguiente:

$$v_1^{**} \approx v_1^* \quad (35)$$

Para calcular v_1^{**} se usa la siguiente ecuación:

$$\begin{bmatrix} v_1^{**} \\ 1 \end{bmatrix} = (K^*)^{-1} \begin{bmatrix} v_{3p} \\ 1 \end{bmatrix} \quad (36)$$

- c) Para este problema se utiliza la función *scipy.optimize.root* con el método *hybr* de la librería Scipy de Python para encontrar la raíz de la función objetivo.
- d) Una vez calculado v_1^* , el punto 2D sin distorsion se calcula de la siguiente manera:

$$\begin{bmatrix} v_{3p} \\ 1 \end{bmatrix} = K^* \begin{bmatrix} v_1^* \\ 1 \end{bmatrix} \quad (37)$$

De esta manera, el punto 2D sin distorsión se puede usar con el modelo de cámara Pinhole ideal.

Por cada cámara calibrada se tiene un modelo de cámara Pinhole ideal que se puede expresar de la siguiente manera utilizando la equ. 1 con $P = KQ$:

$$z_c m = PM \quad (38)$$

$$z_c \begin{bmatrix} x_{3p} \\ y_{3p} \\ z_{3p} \\ 1 \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

A la matriz P se le conoce como matriz de proyección.

Las ecuaciones que genera una sola cámara para la estimación 3D de puntos son las siguientes:

$$\begin{bmatrix} p_{31}x_{3p} - p_{11} & p_{32}x_{3p} - p_{12} & p_{33}x_{3p} - p_{13} \\ p_{31}y_{3p} - p_{21} & p_{32}y_{3p} - p_{22} & p_{33}y_{3p} - p_{23} \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix} = \begin{bmatrix} p_{14} - p_{34}x_{3p} \\ p_{24} - p_{34}y_{3p} \end{bmatrix} \quad (39)$$

Por tanto, para que el sistema de ecuaciones sea candidata a tener una solución, se necesitan mínimo 2 cámaras calibradas. Las ecuaciones resultantes de las cámaras calibradas se deben agregar para formar un sistema de ecuaciones más grande, y además es importante que los puntos 2D sin distorsión de las cámaras sean correspondientes entre sí, es decir, deben corresponder al mismo punto 3D.

El problema se puede plantear y resolver de la siguiente manera:

Problema: $Ax = b$

Solución: $x = (A^T A)^{-1} A^T b$

Donde:

- s : Numero de camaras calibradas ($s \geq 2$)
- $A \in R^{2s \times 3}$
- $b \in R^{2s \times 1}$
- $x = \begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix} \in R^{3 \times 1}$

El análisis anterior es para obtener un solo punto 3D, lo cual da las herramientas para obtener tantos puntos 3D sean necesarios, solo basta con tener puntos 2D sin distorsión y correspondientes entre sí de diferentes cámaras. Un ejemplo de cálculo de puntos 3D se puede ver en la Fig. 11.

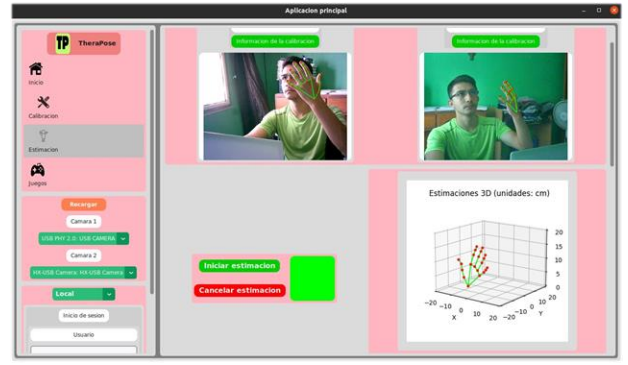


Fig. 11 Puntos 3D a partir de puntos 2D correspondientes dados por el modelo de deep learning *MediaPipe Hands*. Elaboración propia

J. Juegos 2D Implementados

Los juegos 2D desarrollados se crearon en un pequeño framework propio que se desarrolló con la librería CustomTkinter. De esta manera todo el proyecto está únicamente construido en una sola librería, lo que facilita la comunicación entre componentes del proyecto.

Se implementaron 5 juegos interactivos, cada uno de los cuales cumple con alguna tarea en específico en la rehabilitación del paciente. Además es posible configurar ciertos parámetros en cada juego. Un vistazo rápido a los primeros 4 juegos se muestra en la Fig. 12.

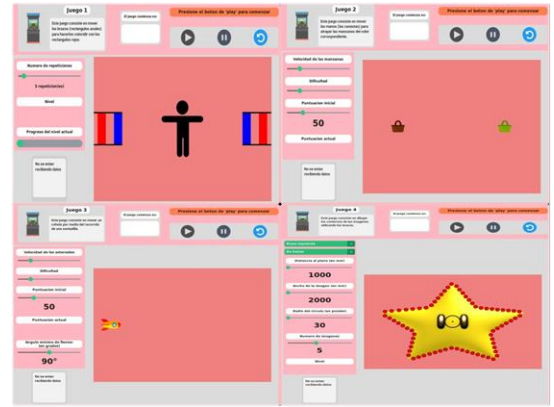


Fig. 12 Juegos 1 al 4 en el orden de izquierda a derecha de arriba a abajo. Elaboración propia

Los juegos están enumerados del 1 al 5 y tratan sobre lo siguiente:

- 1) El juego consiste en replicar la posición de los rectángulos rojos levantando o bajando los brazos, tanto izquierdo como derecho. Los rectángulos azules replican el movimiento de cada brazo de la persona.
- 2) El juego consiste en atrapar manzanas en canastas que se pueden mover con ambas manos. Las

manzanas de un color deben estar en la canasta de ese mismo color.

- 3) El juego consiste en esquivar asteroides moviendo la nave espacial mediante el rango de una sentadilla. La nave espacial estará totalmente controlada por el ejercicio de sentadilla.
- 4) El juego consiste en seguir los contornos de dibujos en pantalla con el brazo (tanto para el brazo izquierdo como para el brazo derecho). Cada imagen es como si se proyectara en grande imaginariamente. Para cada imagen se deben colorear todos los puntos en verde, es decir, se debe pasar por todos los puntos mostrados en la imagen (que en un principio están en color rojo).
- 5) El juego consiste en levantar los dedos de la mano que se piden (ya sea para la mano izquierda o para la mano derecha).

Los juegos del 1 al 4 únicamente necesitan las estimaciones 3D para que el paciente pueda interactuar con los juegos. El juego 5 es algo especial porque no solo necesita de las estimaciones 3D sino que también necesita de clasificadores binarios, uno para cada dedo de las manos, para predecir si el dedo esta levantado o no. Tales clasificadores binarios fueron entrenados en un pequeño framework de deep learning propio. También es propio el conjunto de datos que sirvió de entrenamiento, el cual fue obtenido desde el propio juego 5 (Fig. 13).



Fig. 13 Juego 5. Elaboración propia

Todo el proceso para interactuar con el juego 5 se muestra en la Fig. 14. Los ángulos articulares que se mencionan son los ángulos de Euler de las matrices de rotación de cada uno de los sistemas de coordenadas definidos en cada articulación, donde cada sistema de coordenadas está definido con respecto al sistema de coordenadas anterior.

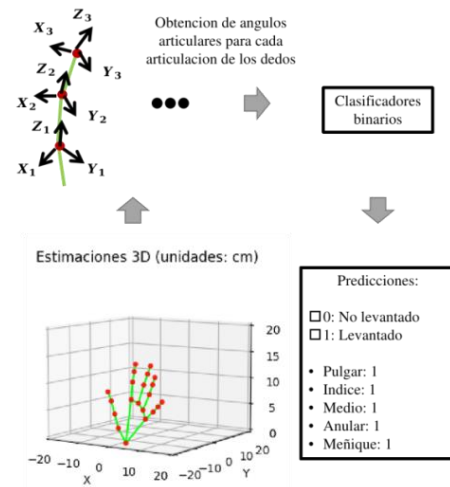


Fig. 14 Proceso para interactuar con el juego 5. Elaboración propia

K. Interacción con Juegos 2D para la Rehabilitación

Para iniciar la interacción con los juegos se debe colocar un patrón conocido de frente a las cámaras ya calibradas para poder iniciar el proceso de estimación de las matrices extrínsecas y de esta manera poder tener las matrices de proyección completas para cada una de las cámaras. Ya teniendo las matrices de proyección es posible realizar las estimaciones 3D.

Para esta tarea el patrón conocido que se suele usar es el marcador ArUco, ya que es fácil y rápido de detectar.

Todo lo que se necesita saber sobre el marcador ArUco se muestra en la Fig. 15.

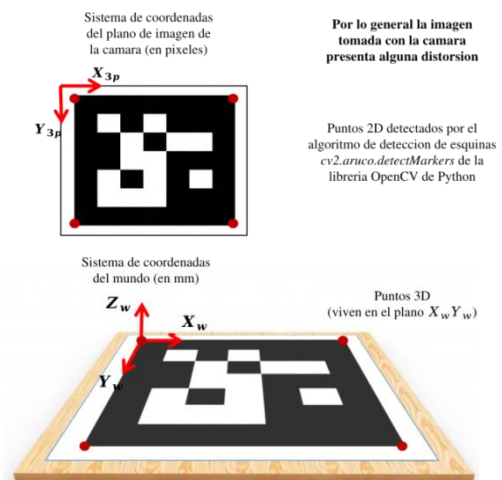


Fig. 15 Obtención de pares de puntos correspondientes 2D y 3D a partir de un marcador ArUco. Elaboración propia

La posición y orientación del marcador ArUco al momento de iniciar las estimaciones 3D es muy importante, ya que de esta depende con que posición y orientación se define el sistema de coordenadas del mundo, la cual pasa además por una serie de transformaciones para quedar orientada de una manera más

cómoda y natural. Las estimaciones 3D se encuentran definidas en este nuevo sistema de coordenadas. En la Fig. 16 se muestra la correcta posición y orientación que debe tener el marcador ArUco.

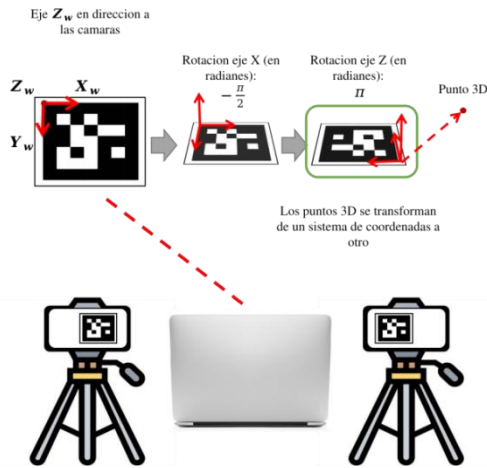


Fig. 16 Posición y orientación del marcador ArUco al momento de iniciar las estimaciones 3D. El sistema de coordenadas con el que se trabaja finalmente es el que se encuentra encerrado en el recuadro de color verde. Elaboración propia

Es importante mencionar que una vez iniciadas las estimaciones 3D las cámaras no se deben de mover. Si por algún motivo las cámaras se mueven, la matriz extrínseca Q previamente calculada para cada cámara ya no será la correcta, ya que la posición y orientación del sistema de coordenadas del mundo con respecto al sistema de coordenadas de cada cámara habrá cambiado, y la matriz extrínseca Q precisamente expresa esa relación. Además, todo esto traerá estimaciones 3D incorrectas, lo cual no es deseable. Por tanto, lo que queda por hacer es repetir el procedimiento para iniciar las estimaciones 3D, asegurando no mover las cámaras en este proceso.

Módulo I Gestión de la tecnología de información

El desarrollo del proyecto está basado en la metodología en espiral, cuenta con una base de datos no relacional MongoDB tanto en local como en la nube y la aplicación Python de escritorio esta desarrollada utilizando el framework CustomTkinter siguiendo buenas prácticas en el diseño y estructuración de la misma, creando componentes reutilizables y minimizando el uso de recursos. Su uso es específico para sistemas operativos Linux que cuentan con Docker y Docker-Compose instalados.

Módulo II Sistemas robustos, paralelos y distribuidos

La aplicación Python de escritorio y la base de datos local MongoDB están desplegadas con Docker-Compose para su fácil distribución y lanzamiento, y la base de datos en la nube esta alojada en el servicio MongoDB Atlas. La base de datos local está configurada como un conjunto de replicas (clúster) con 3 instancias de MongoDB. Para la comunicación entre aplicación y base de datos local se utiliza la red *bridge* de

Docker. El procesamiento intensivo de información como lo son las estimaciones 3D, entre otros, es llevada a cabo mediante hilos para evitar bloqueos en la aplicación.

Módulo III Justificación de Cómputo Flexible (softcomputing)

Se utilizan modelos de deep learning desarrollados en el framework MediaPipe para la detección de puntos característicos del cuerpo humano en imágenes, se tienen clasificadores binarios entrenados en un pequeño framework de deep learning propio, se utiliza minería de datos para la creación y limpieza del conjunto de datos para el entrenamiento de los clasificadores binarios, y se hace uso del modelo de cámara Pinhole (modelo matemático).

IV. RESULTADOS OBTENIDOS DEL PROYECTO

El desempeño del proyecto se basa en lo siguiente:

- 1) La cantidad de pacientes que se pueden tener registrados
- 2) La calidad de las estimaciones 3D y de la interacción con los juegos 2D
- 3) La precisión de los modelos de clasificación binaria para cada uno de los dedos de las manos

Para el punto 1), si no se tenían buenas prácticas de programación y si no se usaba de la manera más eficiente el framework CustomTkinter, cargar toda la información de los pacientes en pantalla era un gran problema y tardaba demasiado, y más si se necesitaban actualizar ciertos componentes ya pintados en pantalla. Como CustomTkinter trata las ventanas como cuadrículas para pintar componentes visuales y todo se pinta una sola vez al iniciar la aplicación, se encontró que es posible acelerar los tiempos de carga y minimizar el uso de memoria si solo se actualizan componentes específicos cuando son requeridos y no componentes grandes que engloban a componentes más pequeños. Para ello se implementaron estrategias para que cada componente tuviera la capacidad de analizar que partes del mismo solo deben actualizarse, es decir, volverse a pintar en pantalla.

Para el punto 2), la calidad de las estimaciones 3D dependen mucho de lo siguiente:

- La cantidad de cámaras que se usan. Entre más cámaras se tengan es mucho mejor, ya que se agregan más ecuaciones (más información) para calcular un punto 3D. Con el mínimo número de cámaras (2) se obtienen resultados buenos.
- Que las cámaras no se muevan una vez iniciadas las estimaciones 3D. Es de mucha importancia que las cámaras se encuentren colocadas en un lugar estable y que sea difícil moverlas.
- Del proceso de refinamiento de los parámetros. La búsqueda de los parámetros óptimos de la cámara debe hacer uso de un algoritmo eficiente (Fig. 17). Tal proceso está planteado como un problema de minimización de una función objetivo, en el cual se

realizan 200 iteraciones utilizando el optimizador *Adam* con las siguientes configuraciones:

- alpha: 0.00001
- beta_1: 0.9
- beta_2: 0.99
- epsilon: 1e-8

Para que la interacción del paciente con los juegos 2D sea de calidad se deben tener estimaciones 3D de calidad y además se debe tener un equipo de cómputo capaz de ejecutar muchos cálculos simultáneamente para que la interacción sea fluida y agradable. Para el equipo de cómputo que se utilizó con procesador Intel i3 8th Gen y con 20GB de RAM, los resultados fueron buenos, logrando fluidez en cada juego.

Para el punto 3), la mejor solución fue crear clasificadores binarios para cada dedo de las manos y no clasificadores multiclase para cada mano. Gracias al paradigma de *divide y vencerás* se optó por este enfoque. Cada clasificador binario fue entrenado en el pequeño framework de deep learning propio con las siguientes características:

- Perceptrón simple
- Inicialización de pesos: Aleatoriamente en el rango de [0,1]
- Función de activación: Sigmoid
- Función de pérdida: Binary Cross Entropy
- Optimizador: Adam
 - alpha: 0.1
 - beta_1: 0.9
 - beta_2: 0.99
 - epsilon: 1e-8
- Métrica de evaluación: F1 Score (para 2 clases)
- Conjunto de datos: Propio

Para cada dedo de cada mano se seleccionaron 3 articulaciones de interés, las cuales generan un total de 9 ángulos por dedo (3 por articulación). Los 3 ángulos por articulación corresponden a los ángulos de Euler de la matriz de rotación de un sistema de coordenadas con respecto a otro.

Aproximadamente se utilizaron entre 150-200 ejemplos para cada clase (dedo levantado y dedo no levantado) con 9 características por ejemplo para formar el conjunto de datos para cada dedo.

- Normalización de datos: Standard Normalization

$$Z = \frac{X - \mu}{\sigma} \quad Z \sim N(0, 1)$$

- División del conjunto de datos
 - 70% entrenamiento
 - 30% prueba
- Numero de épocas: 10
- Tamaño del mini batch: De todos los datos disponibles para el entrenamiento

En la Fig. 18 se pueden ver los resultados obtenidos en la etapa de prueba de los clasificadores binarios, los cuales son muy buenos.

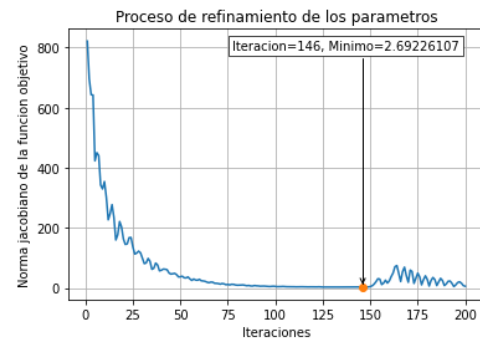


Fig. 17 Los parámetros óptimos en el proceso de refinamiento son aquellos que minimizan la norma del jacobiano (o gradiente en forma de vector fila) en las 200 iteraciones. Debido a que la función objetivo es muy compleja, es muy difícil hacer que el jacobiano sea prácticamente cero. Elaboración propia

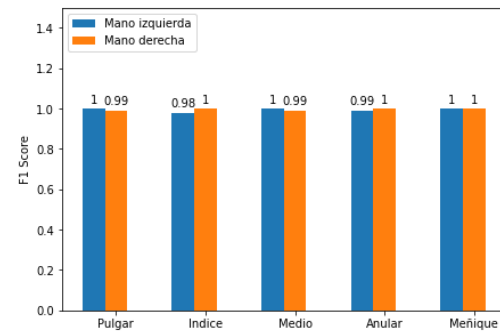


Fig. 18 Resultados arrojados por la métrica de evaluación F1 Score para cada clasificador binario. Elaboración propia

V. CONCLUSIONES Y TRABAJO A FUTURO

El proyecto cumple con servir de sistema informático en la terapia física para el control y registro de pacientes, así como con la implementación de juegos 2D interactivos que ayudan a la rehabilitación del paciente de una manera más controlada y más amena, siendo además muy económico y fácil de usar.

Gracias a este proyecto y a la tecnología actual se abre la posibilidad a muchas alternativas de implementación para mejorar el estado de salud de las personas de una manera más entretenida.

Es posible mejorar el proyecto implementando más variedad de juegos para todo tipo de patologías; guardando los datos recabados por los pacientes de las interacciones con los juegos para analizar su progreso a lo largo del tiempo; creando juegos 3D más reales e interactivos (como en Unity); haciendo uso de realidad virtual para juegos más inmersivos; usando múltiples cámaras para mejorar la calidad de las interacciones; usando sensores corporales para medir fuerzas, velocidades, etc., para obtener datos valiosos del paciente; entre muchas otras mejoras.

RECONOCIMIENTOS

A mi novia María Guadalupe Rodríguez Colin, quien fue parte importante en este proceso, ya que con su gran experiencia en la terapia física logro ayudarme con ideas de ejercicios para

la rehabilitación de pacientes que podían llevarse a cabo a través de juegos interactivos.

A mis padres, quienes siempre me apoyaron en todo este proceso y estuvieron al pendiente de lo que necesitara.

A mi profesor Noé Ortega Sánchez, quien acepto ser mi tutor en este proceso de desarrollo de mi proyecto modular. Siempre se mostró muy atento y amable, y logro aclararme todas las dudas que tuve.

REFERENCIAS

- [1] Artdigital. (s/f). LOKOMAT. Arrayamed.com. Recuperado el 27 de abril de 2024, de <https://www.arrayamed.com/productos/96-lokomat.html>.
- [2] Moro Varas, M. del R. (2012). Aplicaciones de la Wii en Fisioterapia: propiocepción y equilibrio. Universidad de Salamanca.
- [3] Muñoz-Cardona, J. E., Henao-Gallo, O. A., & López-Herrera, J. F. (2013). Sistema de Rehabilitación basado en el Uso de Análisis Biomecánico y Videojuegos mediante el Sensor Kinect. Instituto Tecnológico Metropolitano.
- [4] Pérez Fernández, Á. (2019). Solución de captura de movimiento y seguimiento de articulaciones del cuerpo humano con Kinect 2.0. Universidad Carlos III de Madrid.
- [5] Aguado Fidalgo, C. A. (2015). Realidad Virtual aplicada a la Rehabilitación Física. Universidad Carlos III de Madrid.
- [6] Betances Reinoso, F. A., Lopez Montes, T., Rodríguez Ontiveros, V. M., & Chiesa Estomba, C. (2020). Análisis de la marcha y el equilibrio mediante el uso de sensores inerciales: estudio prospectivo, longitudinal, no aleatorio. *Ciencia y Salud*, 4(1), 11–16. <https://doi.org/10.22206/cysa.2020.v4i1.pp11-16>.
- [7] Vivas Mateos, G. (2016). Desarrollo de un método de captación de movimiento humano para el control remoto de terapias de rehabilitación robóticas. Universidad Politécnica de Madrid.
- [8] Fernández Gómez, R. (2017). Sensor EMG para biofeedback en fisioterapia. Universidad Zaragoza.
- [9] Briseño Cerón, A., Domínguez Ramírez, O. A., & Saucedo Ugalde, I. (2012). El uso de captura de movimiento corporal para el análisis de discapacidades en miembros superior o inferior: Caso de uso: hemiplejía. *TECHNO REVIEW. International Technology, Science and Society Review /Revista Internacional de Tecnología, Ciencia y Sociedad*, 1(2), 31–42. <https://doi.org/10.37467/gka-revtechno.v1.1264>.
- [10] Ramos, J. E. (2015). Desarrollo de un prototipo de sistema de captura de movimiento para actividad física del miembro inferior como interfaz de usuario en un ambiente de realidad virtual. Recuperado de: <http://hdl.handle.net/10654/6921>.
- [11] Mihaiu, C. (s/f). Cosmin Mihaiu. Ted.com. Recuperado el 27 de abril de 2024, de https://www.ted.com/speakers/cosmin_mihaiu.
- [12] Evolv Rehabilitation Technologies. (s/f). Evolvrehab.com. Recuperado el 27 de abril de 2024, de <https://evolvrehab.com/es/>.
- [13] Uhlrich, S. D., Falisse, A., Kidziński, Ł., Muccini, J., Ko, M., Chaudhari, A. S., Hicks, J. L., & Delp, S. L. (2022). OpenCap: 3D human movement dynamics from smartphone videos. En bioRxiv. <https://doi.org/10.1101/2022.07.07.499061>.
- [14] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Ubaweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., & Grundmann, M. (2019). MediaPipe: A Framework for Building Perception Pipelines. <https://doi.org/10.48550/ARXIV.1906.08172>.
- [15] Wikipedia contributors. (s/f). Cámara estenoipeica. Wikipedia, The Free Encyclopedia. https://es.wikipedia.org/w/index.php?title=C%C3%A1mara_estenopei ca&oldid=159432332.
- [16] Puebla, J. (2018). Sistema de detección y caracterización de lanzamientos de un pitcher de béisbol utilizando una cámara PlayStation Eye. Unpublished. <https://doi.org/10.13140/RG.2.2.12213.01763>.
- [17] Zhang, Z. (2000). A flexible new technique for camera calibration. *IEEE transactions on pattern analysis and machine intelligence*, 22(11), 1330–1334. <https://doi.org/10.1109/34.888718>.
- [18] Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., & Grundmann, M. (2020). BlazePose: On-device Real-time Body Pose tracking. <https://doi.org/10.48550/ARXIV.2006.10204>.
- [19] Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., & Grundmann, M. (2020). MediaPipe Hands: On-device real-time hand tracking. <https://doi.org/10.48550/ARXIV.2006.10214>.